

Learned covariance

Completion for missing EEG electrodes

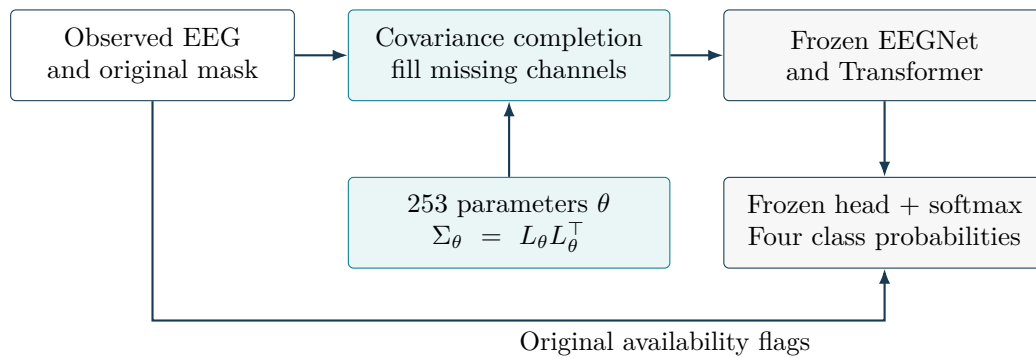
A mathematical and implementation guide

5 October 2026 • Verified CUDA study, version 2

1 What the method does

When an electrode is unavailable, its signal is estimated from the electrodes that remain. A single learned matrix encodes relationships between all 22 channels. The observed subset selects a small linear system; solving it gives the weights used to fill the missing channels. Observed samples are copied unchanged. The completed signal is then classified by an already trained, frozen EEGNet–Transformer.

The trainable component has only **253 scalar parameters**, describing a symmetric positive-definite 22×22 matrix. It is initialized from training signals and adjusted using classification, reconstruction and anchoring losses. There is one fitted matrix per participant and split/training seed, shared across that task’s trials, windows and time samples. It is not recomputed from each evaluation trial, and it is not a Tucker or low-rank tensor decomposition.



The mask also enters the frozen classifier head; predicted samples do not become “observed.” Teal marks the trainable completion component.

Notation used throughout

Symbol	Meaning and dimensions
B, C, P, T	Batch size; $C = 22$ channels; $P = 4$ windows; $T = 250$ samples/window.
X, M	Normalized EEG $X \in \mathbb{R}^{B \times C \times P \times T}$ and Boolean availability $M \in \{0, 1\}^{B \times C \times P}$.
O, H	Observed and hidden channel indices in one trial/window; $H = \{1, \dots, C\} \setminus O$, $ O \geq 1$.
$\tilde{X}$	Completed EEG with the same shape as X .
θ, ϕ	Trainable completion parameters and frozen classifier parameters.

Key point. The model learns how to fill gaps in the classifier’s input. It does not retrain the classifier or modify the samples that were actually observed.

2 From training signals to a valid matrix

2.1 Normalization and initialization

Each four-second trial contains 22 EEG channels sampled at 250 Hz, arranged as four one-second windows. The inherited preprocessing uses 2–30 Hz filtering within each channel/window. Let R denote these filtered signals. For each channel c , compute the mean μ_c and population standard deviation s_c using *training trials and all their time samples only*. Then

$$X_{bcpt} = \frac{R_{bcpt} - \mu_c}{\max(s_c, 10^{-8})}. \quad (1)$$

The same constants are used for validation and later inference. This removes a training channel mean, not a separate mean for every trial or window.

Collect all $n = N_{\text{train}}PT$ training-time channel vectors into $Q \in \mathbb{R}^{n \times C}$. Initialize from the uncentered second moment of the already normalized data:

$$S = \frac{1}{n}Q^\top Q, \quad \Sigma_0 = \frac{1}{2}(S + S^\top) + \varepsilon I_C, \quad \varepsilon = 10^{-6}. \quad (2)$$

Training centering makes S approximately the population covariance, up to finite precision. There is no extra per-trial centering or $n - 1$ correction. The diagonal jitter ε permits a stable Cholesky initialization. It is distinct from the solve regularization introduced on the next page.

2.2 Positive definiteness by construction

A general symmetric matrix can become indefinite during optimization. Instead, store a vector $\theta \in \mathbb{R}^{C(C+1)/2} = \mathbb{R}^{253}$ containing the free lower-triangle entries. Define

$$(L_\theta)_{ij} = \begin{cases} \theta_{ij}, & i > j, \\ \text{softplus}(\theta_{ii}) + \delta, & i = j, \\ 0, & i < j, \end{cases} \quad \Sigma_\theta = L_\theta L_\theta^\top, \quad \delta = 10^{-6}, \quad (3)$$

where $\text{softplus}(z) = \log(1 + e^z)$, evaluated with a numerically stable routine. The positive diagonal makes L_θ invertible. Therefore, for every nonzero v ,

$$v^\top \Sigma_\theta v = \|L_\theta^\top v\|_2^2 > 0. \quad (4)$$

This is a full-rank matrix factorization, not a dimensionality reduction. No orthogonality, unit-column projection or unit-diagonal constraint is applied.

To match the initial covariance, compute $L_0 = \text{chol}(\Sigma_0)$. Copy its off-diagonal entries and initialize its diagonal free parameters by

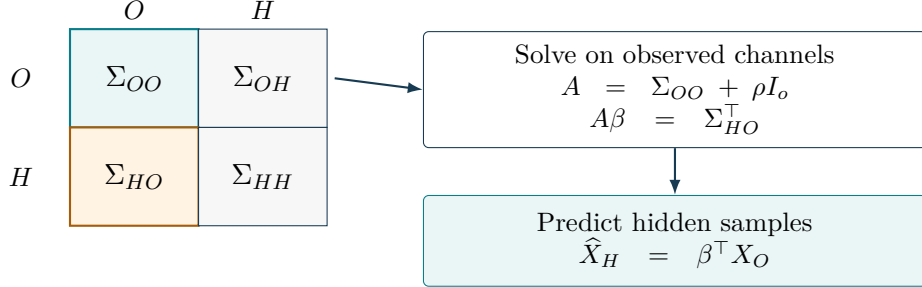
$$z_i = (L_0)_{ii} - \delta, \quad (\theta_0)_{ii} = z_i + \log(1 - e^{-z_i}). \quad (5)$$

Here $z_i > 0$ is checked. The expression is the inverse softplus, evaluated stably in code. Thus both covariance arms start with the same Σ_0 up to floating-point precision. The frozen arm retains θ_0 ; the learned arm optimizes θ and keeps a separate, immutable copy of θ_0 .

Key point. Only the training set determines initialization. Positive definiteness is maintained structurally, while the matrix’s entries can adapt to the task.

3 Completing one partially observed window

For one trial/window, write $X_O \in \mathbb{R}^{o \times T}$ for its observed rows, where $o = |O|$, and let $h = |H|$. Original channel identities determine the rows and columns extracted from $\Sigma = \Sigma_\theta$; channel order is never lost.



Conceptual block permutation for explanation only. The implementation indexes the original channel coordinates and restores predictions to those coordinates.

3.1 The regularized conditional predictor

Set a scale-adaptive ridge coefficient using the *whole* matrix:

$$\rho(\Sigma) = \alpha \frac{\text{tr}(\Sigma)}{C}, \quad \alpha = 10^{-3}. \quad (6)$$

The missing-channel estimate is

$$\hat{X}_H = \Sigma_{HO}(\Sigma_{OO} + \rho I_o)^{-1} X_O. \quad (7)$$

Here Σ_{HO} has shape $h \times o$. Code does not form an inverse: it solves $A\beta = \Sigma_{HO}^\top$ with $\beta \in \mathbb{R}^{o \times h}$ and multiplies $\beta^\top X_O$. One solve supplies the weights for all 250 time samples, and trial/windows with identical masks are grouped.

For nonzero $u \in \mathbb{R}^o$, $u^\top A u = u^\top \Sigma_{OO} u + \rho \|u\|^2 > 0$. The system consequently has a unique solution in exact arithmetic. Covariance construction and solves use float64, with finite-value checks; the predictions are cast back to the signal dtype. Positive definiteness alone does not promise perfect numerical conditioning for arbitrary parameters.

3.2 Why this expression is reasonable

Under a zero-mean Gaussian model, the conditional mean of hidden coordinates is $\Sigma_{HO} \Sigma_{OO}^{-1} x_O$ [1]. Adding independent observation noise with variance ρ gives Equation (7). This is a motivation for the regularized predictor, not an assertion that actual EEG is Gaussian or that its noise variance was estimated. The implementation retains the observed values exactly, rather than denoising them under that model.

Finally, fill only hidden entries:

$$\tilde{X}_{c,t} = \begin{cases} X_{c,t}, & c \in O, \\ \hat{X}_{c,t}, & c \in H. \end{cases} \quad (8)$$

An all-observed window needs no prediction. An all-missing window is rejected: this method requires at least one observed electrode.

4 A worked example and useful properties

4.1 Three channels, one missing

Consider the illustrative positive-definite matrix

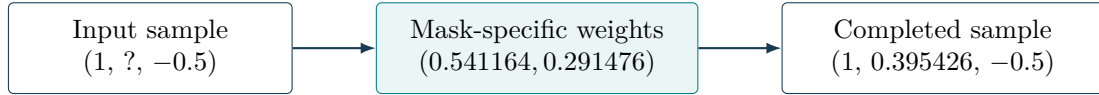
$$\Sigma = \begin{pmatrix} 1 & 0.6 & 0.2 \\ 0.6 & 1 & 0.4 \\ 0.2 & 0.4 & 1 \end{pmatrix}, \quad x_O = \begin{pmatrix} 1 \\ -0.5 \end{pmatrix}, \quad O = \{1, 3\}, \quad H = \{2\}. \quad (9)$$

The trace rule gives $\rho = 0.001$. Therefore

$$A = \begin{pmatrix} 1.001 & 0.2 \\ 0.2 & 1.001 \end{pmatrix}, \quad \Sigma_{HO} = \begin{pmatrix} 0.6 & 0.4 \end{pmatrix}, \quad (10)$$

$$\beta = A^{-1} \begin{pmatrix} 0.6 \\ 0.4 \end{pmatrix} \simeq \begin{pmatrix} 0.541164 \\ 0.291476 \end{pmatrix}, \quad \hat{x}_2 = 0.541164(1) + 0.291476(-0.5) \simeq 0.395426. \quad (11)$$

The completed vector is $(1, 0.395426, -0.5)^\top$. The first and third samples are copied, not re-estimated. This toy calculation is not a measured EEG result; it shows the arithmetic used at each time sample.



Availability remains $(1, 0, 1)$.

4.2 What is shared and what changes

Quantity	Dependence
Σ_θ	Shared over every trial, window and sample in one fitted task.
O, H	Determined by the known electrode mask for each trial/window.
$W_{H \leftarrow O} = \beta^\top$	Depends on the mask and Σ_θ , but not on signal amplitudes.
$\hat{X}_H$	Depends linearly on the observed signal through $W_{H \leftarrow O} X_O$.

For a fixed mask, this is an instantaneous spatial linear predictor. It mixes electrodes at the same sample time, independently within each window. It does not learn a temporal filter or use another window to infer the missing signal. The downstream EEGNet–Transformer still models time and can mix windows.

The weights need not be positive and need not sum to one. Thus this is neither a convex average nor distance-based interpolation. One shared SPD matrix couples the predictors for all masks; they are not independently trained regression heads for every possible subset.

The prediction weights are invariant to a positive global scale of Σ :

$$W_{H \leftarrow O}(a\Sigma) = a\Sigma_{HO}(a\Sigma_{OO} + a\rho(\Sigma)I_O)^{-1} = W_{H \leftarrow O}(\Sigma), \quad a > 0. \quad (12)$$

This follows specifically from the trace-scaled ridge. The parameter anchor still depends on scale through θ , so the entire training objective is not scale-invariant.

Key point. The learned object is a compact, shared family of mask-dependent linear completion rules. Its usefulness must be judged by held-out prediction, not by treating matrix entries as physiological connectivity.

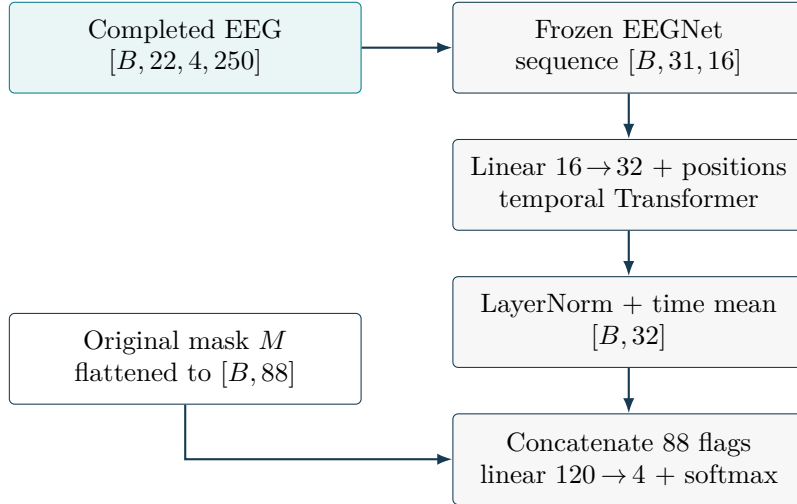
5 Connecting completion to a frozen classifier

5.1 Forward route and availability flags

Let g_ϕ denote the historical EEGNet–Transformer computation applied to *completed* signals, with original mask flags appended at the output head. The adapted predictor is

$$p_\theta(y \mid X_O, M) = \text{softmax}(g_\phi(\tilde{X}_\theta, M)), \quad \phi \text{ fixed.} \quad (13)$$

The historical backbone uses an EEGNet front end, inspired by the compact EEG architecture of Lawhern et al. [2], followed by a temporal Transformer. The exact local route is shown below.



The Transformer has one pre-normalized layer, four heads, feed-forward width 64, GELU and fixed sinusoidal positions. The EEGNet front end concatenates the four windows into 1,000 samples for its inherited convolutions; only completion and preprocessing are window-local. All backbone weights and BatchNorm statistics remain fixed, and dropout remains disabled.

Do not mask the completed signal back to zero. The adapter feeds filled values directly into the existing layers. It retains M for the 88 head features. Calling an ordinary raw-masking backbone route on $(\tilde{X}, M)$ would erase the completion. Entirely full-input batches bypass the completer and call the original model exactly, preserving full-input predictions across strategies.

5.2 A frozen network still transmits input gradients

Freezing ϕ means not updating it; it does not mean wrapping its training forward pass in a no-gradient context. Classification derivatives must reach $\tilde{X}_\theta$ and then θ . To see the solve derivative, equivalently write $AZ = X_O$, $\hat{X}_H = \Sigma_{HO}Z$. For fixed observed input,

$$dZ = -A^{-1}(dA)Z, \quad dA = d\Sigma_{OO} + d\rho I_o, \quad (14)$$

$$d\hat{X}_H = (d\Sigma_{HO})Z - \Sigma_{HO}A^{-1}(dA)Z, \quad d\rho = \frac{\alpha}{C} \text{tr}(d\Sigma). \quad (15)$$

Autograd differentiates the solve and the Cholesky parameterization, including the dependence of ρ on the learned diagonal. No eigendecomposition or explicit inverse is differentiated.

6 Learning objective and optimization

Training starts with complete training trials. Artificial masks hide some electrodes from the forward computation, while their true samples remain available only as reconstruction targets. For a batch of size B , labels $y_b \in \{0, 1, 2, 3\}$, and scalar hidden-sample index set $\mathcal{H} = \{(b, c, p, t) : M_{bcp} = 0\}$, define

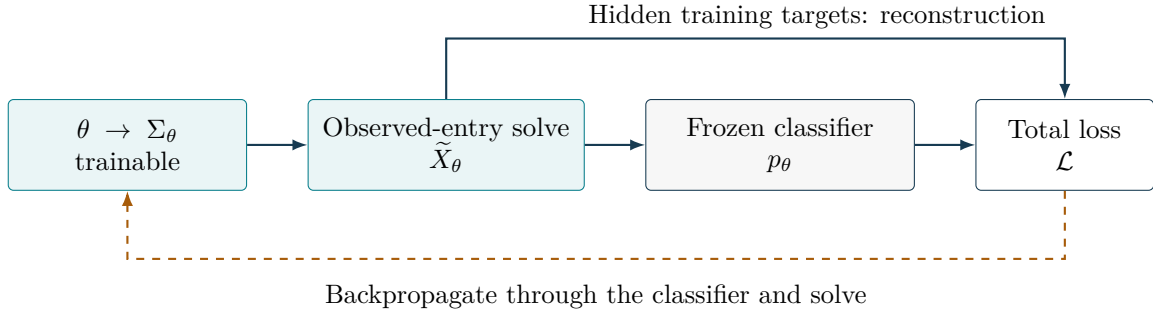
$$\mathcal{L}_{\text{cls}} = -\frac{1}{B} \sum_{b=1}^B \log p_{\theta}(y_b \mid X_{b,O}, M_b), \quad (16)$$

$$\mathcal{L}_{\text{rec}} = \frac{\sum_{(b,c,p,t) \in \mathcal{H}} (\tilde{X}_{bcpt} - X_{bcpt})^2}{\max(1, |\mathcal{H}|)}, \quad (17)$$

$$\mathcal{L}_{\text{anchor}} = \frac{1}{253} \|\theta - \theta_0\|_2^2, \quad (18)$$

$$\boxed{\mathcal{L} = \mathcal{L}_{\text{cls}} + 0.1\mathcal{L}_{\text{rec}} + 10^{-4}\mathcal{L}_{\text{anchor}}.} \quad (19)$$

The reconstruction denominator counts hidden scalar samples over the entire batch. Observed entries do not enter that error. The anchor is on the free Cholesky parameters, not on Σ itself. It discourages large departures from the training-statistics initialization. No Gaussian likelihood is optimized.



One training run

1. Load the historical participant/seed split and frozen classifier. Verify its original full-input predictions. Fit normalization and Σ_0 on training data.
2. At each epoch, draw paired per-trial masks: retain 22, 16 or 6 electrodes with equal probability. For nonfull input, static/dynamic random loss is equally likely. Preserve channel identities and partition-disjoint mask subsets.
3. Compute Equation (19), backpropagate only to θ , clip its gradient norm at 1, and take an Adam step. Keep the backbone in evaluation mode. Wholly full batches skip the optimizer step, including any anchor-only update.
4. Evaluate the fixed degraded validation bank. Retain the best checkpoint; restore its numeric parameters after training. No test targets enter this loop.

Optimizer	Adam; learning rate 10^{-3} ; batch size 32; zero weight decay.
Budget	At most 100 epochs; at least 10 before stopping; patience 20.
Selection	Highest mean degraded validation balanced accuracy; ties: lower log loss, then earlier epoch. Epoch zero is eligible.
Additional settings	No scheduler, no mixed precision; only completion parameters update.

7 What the completed experiment shows

7.1 Paired validation protocol

The study uses nine BCI IV 2a participants, T-session data, and seeds 0, 1 and 2. Each seed defines a participant-specific, approximately 60/20/20 train/validation/test split inherited from the earlier study; this screen uses training and validation only. Seeds vary splits as well as training randomness. There are 27 tasks and 54 supervised fits: learned covariance and learned Tucker in each task. Zero, frozen covariance and frozen Tucker are paired controls; no backbone is retrained.

Evaluate all 22 electrodes once, plus static and dynamic random retention of 16 and 6 electrodes, each with five paired mask repeats. Static masks retain one subset across all windows. The implemented dynamic schedule uses subsets A, B, A, A over the four windows, with $A \neq B$ and equal retained counts. Completion selection and reported degraded evaluation use the same masks.

For each cell, balanced accuracy is the average of the four class recalls:

$$\text{BA} = \frac{1}{4} \sum_{k=0}^3 \frac{\sum_b \mathbf{1}[y_b = k, \arg \max_j p_{bj} = k]}{\sum_b \mathbf{1}[y_b = k]}. \quad (20)$$

Average mask repeats, then seeds within a participant, then participants equally. The overall degraded score also averages the four degraded conditions equally. The 16- and 6-electrode columns below each average static and dynamic conditions.

7.2 Verified cohort results

Completion	Overall BA (%)	16 retained BA (%)	6 retained BA (%)	Log loss overall
Zero filling	34.17	36.68	31.66	2.317
Frozen Tucker	36.01	40.12	31.91	2.178
Learned Tucker	38.45	41.87	35.04	2.049
Frozen covariance	49.57	58.27	40.87	1.465
Learned covariance	55.92	59.81	52.04	1.149

Log loss uses natural logarithms and lower is better. All strategies have identical full-input predictions: cohort balanced accuracy is 64.87%. Completion has no full-input accuracy-gain objective.

Learned covariance improves on frozen covariance by **6.35 percentage points**, with positive participant averages in 9/9 participants. The exploratory 95% participant-bootstrap interval is [4.01, 9.45] points (2,000 resamples, seed 20261005). The gain is 1.54 points with 16 electrodes and 11.17 points with 6 electrodes. These condition gains are descriptive, not separate confirmatory hypothesis tests.

Learned Tucker gains 2.44 points over frozen Tucker, meeting the declared primary exploratory screen (≥ 2 points and $\geq 6/9$ positive participants). Nevertheless, learned covariance exceeds learned Tucker by 17.47 points. Covariance comparisons are secondary contrasts in the original declaration; the primary endpoint has not been changed retrospectively.

Key point. This screen favors learned covariance as the candidate for an independent follow-up. It supports a practical choice within this tested pipeline, not a universal claim about covariance versus tensor models.

8 Interpretation, safeguards and reproduction

8.1 What the evidence can and cannot establish

After supervised updates, Σ_θ parameterizes useful completion rules. It is not a calibrated physiological covariance or causal connectivity estimate. The model supplies point predictions, without validated uncertainty estimates.

Both backbone and completion checkpoints used validation selection. Epoch zero reproduces the frozen initialization and is eligible, so the selected model cannot have a lower *selection score* than that initialization. Nonnegative gains on this bank are partly built into selection. Bootstrap intervals do not correct selection optimism, validation reuse or multiple comparisons. Independent evaluation with settings frozen is still required.

Simulated missingness on complete trials does not establish robustness to corrupt observed channels, missing calibration channels, unseen participants or new sessions. Why Tucker underperforms is unresolved; different ranks, losses or backbones require new experiments.

8.2 Implementation safeguards

- Select hidden NaN/Inf values away with `torch.where` before arithmetic; zero multiplication cannot safely remove NaNs. Reject observed nonfinite samples and all-missing windows. Preserve observed samples and original mask flags.
- Calibration stays on CPU; optimization and prediction use CUDA. The verified execution disables TF32, autocast and fused MHA fastpath, with deterministic algorithms. CPU/GPU full-input tolerance is $\text{rtol} = 10^{-4}$, $\text{atol} = 10^{-6}$; same-device saved-state replay is exact.

8.3 Source and evidence map

Definition	Local source or artifact
Covariance and masking	<code>inm/task_driven_completion/completion.py</code>
Differentiable backbone adapter	<code>inm/task_driven_completion/adapters.py</code>
CUDA training and audit	<code>inm/task_driven_completion_cuda/</code>
Frozen declaration	<code>configs/task-driven-completion-cuda-v2.json</code>
Measured tables and provenance	<code>results/task-driven-completion-cuda-v2/report/</code>
Independent completion review	<code>.session-runs/task-driven-completion-cuda/</code> <code>completion-verification/verification.json</code>

All 27 tasks / 54 supervised fits completed. Independent GPU audits exactly replayed 2,835 cells; a subsequent review verified hashes and recomputed metrics, aggregation and intervals. Study identity:

74bfeb36445f3b0d365fac8ca03514e9e8e359c2c205b277e0389bf2922eb071

Build: `make -C docs/concept learned_covariance.pdf`. Diagrams are editable TikZ vectors; no external images are needed.

References

- [1] C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006, Appendix A.2. [Gaussian conditioning and matrix identities](#).
- [2] V. J. Lawhern et al. *EEGNet: A Compact Convolutional Network for EEG-based Brain-Computer Interfaces*. [arXiv:1611.08024](#).